

BLOC 3 - FRAMEWORK

Developpement avance : comprendre et utiliser une architecture de type framework

Ce que le jury attendait sur chaque competence, et ce que j'ai fait pour y repondre.

Framework utilise : Flask (Python)

Projet Airlines Reservation - Flask + PostgreSQL

Document de preparation orale au jury

Retour du premier passage

CE QUI ETAIT ATTENDU

"Le framework est compris et bien pris en main. Le projet est fonctionnel, mais pourrait être davantage approfondi, notamment avec la mise en place d'un espace d'administration et de fonctionnalités complètes de gestion des données : ajout, modification (UPDATE) et suppression (DELETE)."

CE QUE J'AI FAIT

C'était déjà le bloc le mieux noté au premier passage (67/100), les 3 compétences étaient acquises à 2/3 chacune. Le seul manque concret pointé par le jury - l'absence d'espace d'administration avec un vrai CRUD - est traité dans la compétence C5.c ci-dessous.

C5 - Framework (Flask)

C5.a Confronter ses connaissances avec la documentation du framework pour s'approprier son architecture

CE QUI ETAIT ATTENDU

Que je ne me contente pas d'utiliser Flask de façon basique, mais que je combine plusieurs mécanismes du framework en m'appuyant sur sa documentation officielle.

CE QUE J'AI FAIT

J'utilise trois extensions officielles de l'écosystème Flask, chacune configurée selon sa propre documentation : Flask-Login pour les sessions et l'authentification, Flask-WTF pour la protection CSRF, et Flask-Limiter pour limiter le nombre de requêtes. Pour aller plus loin, j'ai construit un décorateur maison, `admin_required`, qui combine le décorateur natif `@login_required` de Flask-Login avec ma propre vérification de rôle (`current_user.is_admin`). Ce n'est pas juste copier un exemple de la doc : il faut comprendre précisément comment Flask-Login gère `current_user` pour pouvoir l'étendre correctement. J'ai aussi mis en place un gestionnaire d'erreur personnalisé qui intercepte le code HTTP 429 (trop de requêtes) généré automatiquement par Flask-Limiter, pour rediriger l'utilisateur avec un message clair au lieu de lui montrer la page d'erreur brute par défaut.

```
-> app.py, ligne 75 : def admin_required(f): ... @login_required ...  
-> app.py, ligne 37 : @app.errorhandler(429)
```

C5.b Configurer le framework avec un gestionnaire de dépendances

CE QUI ETAIT ATTENDU

Que les dépendances du projet et la configuration sensible soient gérées proprement, pas codées en dur.

CE QUE J'AI FAIT

Toutes mes dependances sont gérées par pip avec un fichier requirements.txt, et je fige les versions précisément (Flask==3.1.3, Flask-Login==0.6.3, etc.) pour garantir que l'environnement d'installation reste identique et compatible dans le temps. Toute la configuration sensible - connexion à la base de données, clé secrète de session, paramètres SMTP pour l'envoi d'email - est centralisée dans un fichier .env, jamais committé sur GitHub, et chargée au démarrage via python-dotenv. Cela sépare clairement la configuration sensible du code source : 14 variables d'environnement au total.

```
-> requirements.txt : Flask==3.1.3, Flask-Limiter==4.1.1, Flask-Login==0.6.3,
Flask-WTF==1.3.0, psycpg2-binary==2.9.12, python-dotenv==1.2.2
```

```
-> Database/db.py, ligne 3 : from dotenv import load_dotenv ; load_dotenv()
```

C5.c

Développer une application évolutive avec le framework

Corrige - réponse directe au point soulevé par le jury

CE QUI ÉTAIT ATTENDU

Exactement le point soulevé par le jury : mettre en place un espace d'administration avec des fonctionnalités complètes de gestion des données (ajout, modification, suppression), et montrer une application capable d'évoluer, d'être déboguée et livrée fonctionnelle.

CE QUE J'AI FAIT

J'ai construit un espace d'administration complet, avec une mise en page séparée du site client (une sidebar dédiée, sans header ni footer commercial), qui répond exactement au manque signalé : lister les comptes clients, en créer un nouveau, modifier un compte existant, et en supprimer un - les 4 routes correspondantes. Un administrateur est automatiquement redirigé vers ce dashboard dès sa connexion, sans que cela change quoi que ce soit au parcours des clients classiques. Toutes ces routes sont protégées par le décorateur admin_required vu au C5.a : je l'ai vérifié avec de vraies requêtes HTTP, en utilisant deux comptes distincts, pas seulement en relisant le code - un visiteur non connecté est redirigé vers la connexion, un client connecté mais non-admin reçoit une erreur 403. Pendant le développement, j'ai aussi rencontré et corrigé un vrai bug : l'ordre des colonnes renvoyées par une requête SQL ne correspondait plus à l'ordre attendu par mon code Python, ce qui aurait pu faire apparaître n'importe quel utilisateur comme administrateur. Je l'ai corrigé définitivement en passant à un curseur qui renvoie des dictionnaires nommés plutôt que des tuples positionnels, pour éliminer ce risque d'erreur silencieuse. Enfin, l'application entière repose sur une gestion d'erreurs systématique - 25 blocs try/except et 23 blocs finally dans app.py - qui garantit qu'une connexion à la base est toujours fermée même en cas d'erreur, et que je ne montre jamais une trace technique au visiteur.

```
-> app.py, ligne 91 : GET      /admin/comptes
-> app.py, ligne 133 : POST    /admin/api/comptes
-> app.py, ligne 165 : PUT      /admin/api/comptes/<id>
-> app.py, ligne 211 : DELETE  /admin/api/comptes/<id>
-> app.py : cursor_factory=RealDictCursor - accès aux colonnes par nom (user['email'])
```

